

Scalable Network I/O

`select` → `poll` → `epoll` → `io_uring`

Four single-threaded TCP echo servers, benchmarked from 10 to 1,000 concurrent connections — now with true per-request latency percentiles, measured CPU/memory efficiency, and a documented hardware environment closing the gaps from our first submission.

Kale Parth Hemant *select*

Patil Nachiket Kiran *epoll*

Chirag Agarwalla *Benchmarking*

Devvrat Hans *poll*

Niraj Kumar *io_uring*

Nityanand Karmali *Report*



What we set out to build and prove

Implement select, poll, epoll, and io_uring from scratch as single-threaded TCP echo servers, then benchmark and profile them under rising connection load — throughput, latency, syscall frequency, and CPU/memory efficiency — to explain why readiness-notification designs degrade under concurrency and how epoll and io_uring change that picture.

Expected outcomes

- ◆ Four working C engines: select, poll, epoll, io_uring
- ◆ Load-varied benchmarking (10 / 100 / 1,000 connections)
- ◆ Syscall profiling, true latency percentiles, CPU % and RSS
- ◆ A written analysis of $O(N)$ vs. $O(\text{ready})$ vs. $O(1)$ bottlenecks

Six research questions frame the study

RQ1

How do throughput and latency change from 10 → 100 → 1,000 connections?

RQ2

How do event-wait and syscall frequencies change with load?

RQ3

Why is select/poll $O(N)$, epoll $O(\text{ready})$, and io_uring $O(1)/\text{batched}$?

RQ4

Does io_uring's ring design measurably cut transitions vs. epoll?

RQ5

How do CPU utilization and memory footprint scale with load?

RQ6

What trade-offs remain even in the fastest mechanisms?

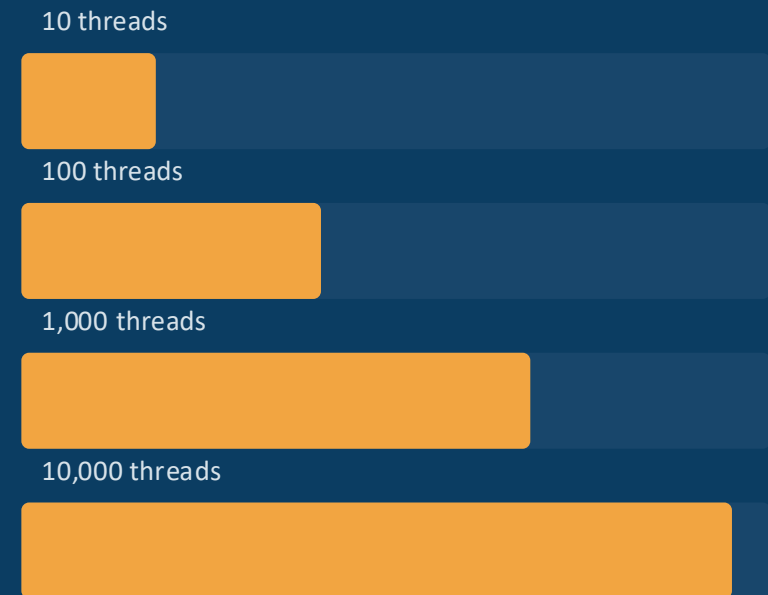
One thread per connection does not scale

A thread-per-connection server gives every client its own blocked thread. Each thread carries a fixed kernel stack and scheduling weight, so memory footprint grows with the number of connections, not with how much work is actually happening. Past a few thousand simultaneous clients, the OS scheduler spends more time context-switching between mostly idle threads than serving any of them — the classic C10K problem.

I/O multiplexing replaces this with a single thread that asks the kernel which of many watched sockets are actually ready, and only acts on those. It trades per-connection threads for one event loop — the foundation every mechanism in this project builds on.

Scheduling + memory pressure

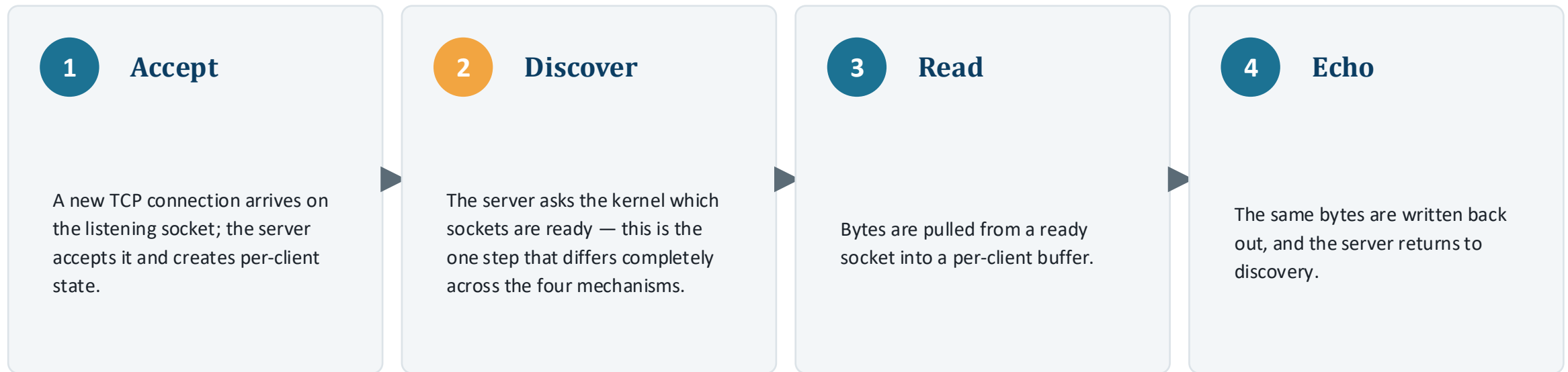
thread-per-connection model (illustrative)



Illustrative relative cost — not measured data.

The single-threaded TCP echo server

All four engines answer the same brief: one OS thread, listening on port 9090, holding per-client state indexed by file descriptor, echoing every byte it receives back to the sender. None of them ever blocks on a single client's socket — the whole point of the exercise is to find out how each engine discovers, cheaply, which of its many sockets are ready to be read from or written to right now.



Everything from here on is about what happens in step 2 — discovery — for each of the four mechanisms.

select()

The original multiplexing call — and the one with a hard descriptor ceiling



`select()` asks the application to describe its entire interest set on every single call: two bitmaps — one for read interest, one for write — are rebuilt from the live client list each loop iteration, because the kernel destroys and overwrites them in place before returning.

The kernel then walks every bit from 0 up to the highest file descriptor currently in use, whether or not that descriptor belongs to an active connection, and the application walks the returned bitmap again to find out which bits actually fired. Cost tracks the highest fd value in play, not the number of live clients — a purely structural $O(\text{max_fd})$ tax paid on every wait, win or lose.

- **Hard ceiling:** `FD_SETSIZE` is fixed at 1024 descriptors. At the 1,000-connection test point this implementation is operating close to that architectural boundary, not comfortably inside it.

poll()

Drops the bitmap ceiling, keeps the $O(N)$ scan



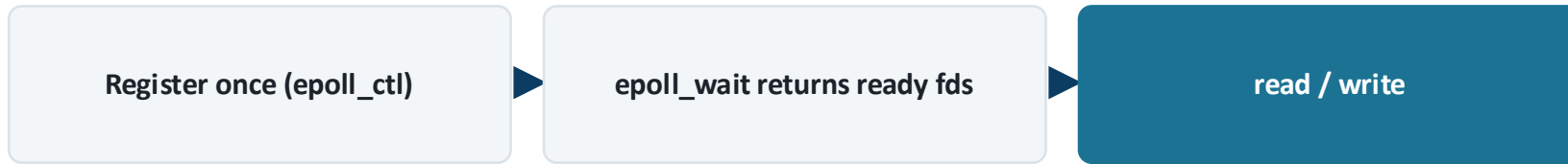
`poll()` replaces the fixed-size bitmap with a flat, dynamically sized array of `pollfd` structures — one entry per watched socket, each carrying its own requested and returned event flags. This removes the hard descriptor ceiling entirely: the array can grow as large as the process's file-descriptor limit allows.

But the underlying cost shape does not change. Every call still means handing the kernel the full array, having the kernel inspect every entry, and having the application walk the same array again afterward to find which revents fields are set — an $O(N)$ scan over the whole watched set on every iteration, active or not, just without `select`'s bitmap cap.

- **Deferred write:** `POLLOUT` is only armed once a read leaves data still unsent, so the write happens one loop iteration after the read that produced it — a scheduling detail that matters when comparing syscall counts against `epoll`.

epoll

Registration moves into the kernel — the app stops re-describing its interest set



epoll splits registration from waiting. An interest list is created once with `epoll_create1()` and lives inside the kernel across every subsequent call — there is no bitmap or array to rebuild and hand over each loop. Adding or removing a socket costs one `epoll_ctl()` call, paid only when a connection actually opens or closes, not on every wait.

`epoll_wait()` then returns only the descriptors that are genuinely ready — nothing is scanned that isn't active. Per-call cost scales with the number of ready events, $O(\text{ready})$, completely decoupled from how many thousands of sockets are simply sitting idle in the interest list. This implementation reads and, in the same dispatch, immediately attempts the echo write before returning to `epoll_wait()`.

- **Fairness caveat:** Writing within the same event-loop pass (rather than deferring it, as `select/poll` do) can favour epoll's measured throughput and syscall counts — a real implementation asymmetry, stated rather than hidden.

io_uring

Changes the model entirely: from readiness notification to completion processing



io_uring abandons readiness notification altogether. The application places whole operations — accept, read, write — into a shared submission queue (SQ); the kernel then performs those operations asynchronously, without the application ever calling `read()` or `write()` itself for each one.

Completions are posted to a completion queue (CQ) and drained in a single batch per `io_uring_enter()` call, so an entire burst of finished I/O is collected through one user/kernel crossing instead of one crossing per operation. This implementation starts a multishot accept once at startup and keeps exactly one I/O operation outstanding per client, guarded explicitly so a second op is never submitted while one is already in flight.

● **Trade-off:** One outstanding operation per client is a deliberate simplification — it caps pipeline depth and leaves some of io_uring's batching potential on the table in this implementation.

Two measurement passes, one documented machine

① Core benchmark

throughput · latency · syscalls — C client + strace

Connections	10, 100, 1,000
Client workers	4 processes
Messages / connection	100
Payload	64 bytes, request + reply
Repetitions	5 per config, warm-up excluded
Timing	CLOCK_MONOTONIC, loopback

② Supplementary profiling pass

CPU % · peak RSS · true percentiles — /usr/bin/time -v + asyncio client

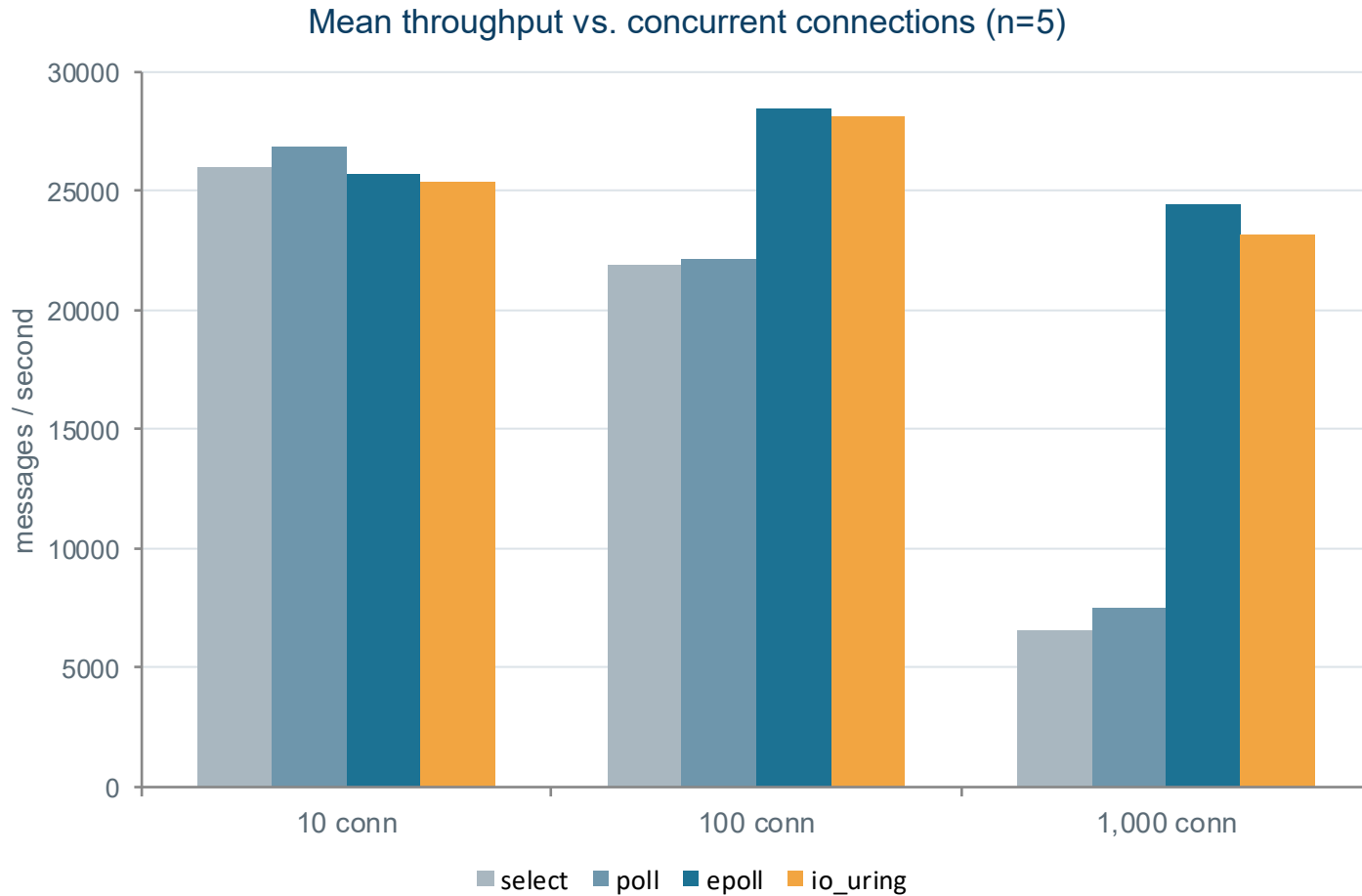
Connections	10, 100, 1,000
Client	1 async process (latency_profiler.py)
Messages / connection	10
Repetitions	1 — indicative, not averaged
CPU / memory	/usr/bin/time -v per (engine, conn)
Latency	every request timed, sorted → percentiles

Environment (both passes): Apple M2 (Avalanche-M2), 4 cores · 7.3 GiB RAM · Fedora Linux, kernel 7.1.6-400.asahi.fc44.aarch64 · gcc 16.2.1

Correctness validated byte-for-byte across all 60 core-benchmark runs (4 engines × 3 levels × 5 reps) — zero content mismatches. A fairness re-test standardized select and poll to infinite blocking, matching epoll/io_uring and removing the idle-timeout confound.

RESULTS

Throughput holds under epoll & io_uring — not select/poll



6.6k ▶ 24.4k

msg/s at 1,000 connections: select vs. epoll — a 3.7× gap that opens up only under load.

28.1–28.4k

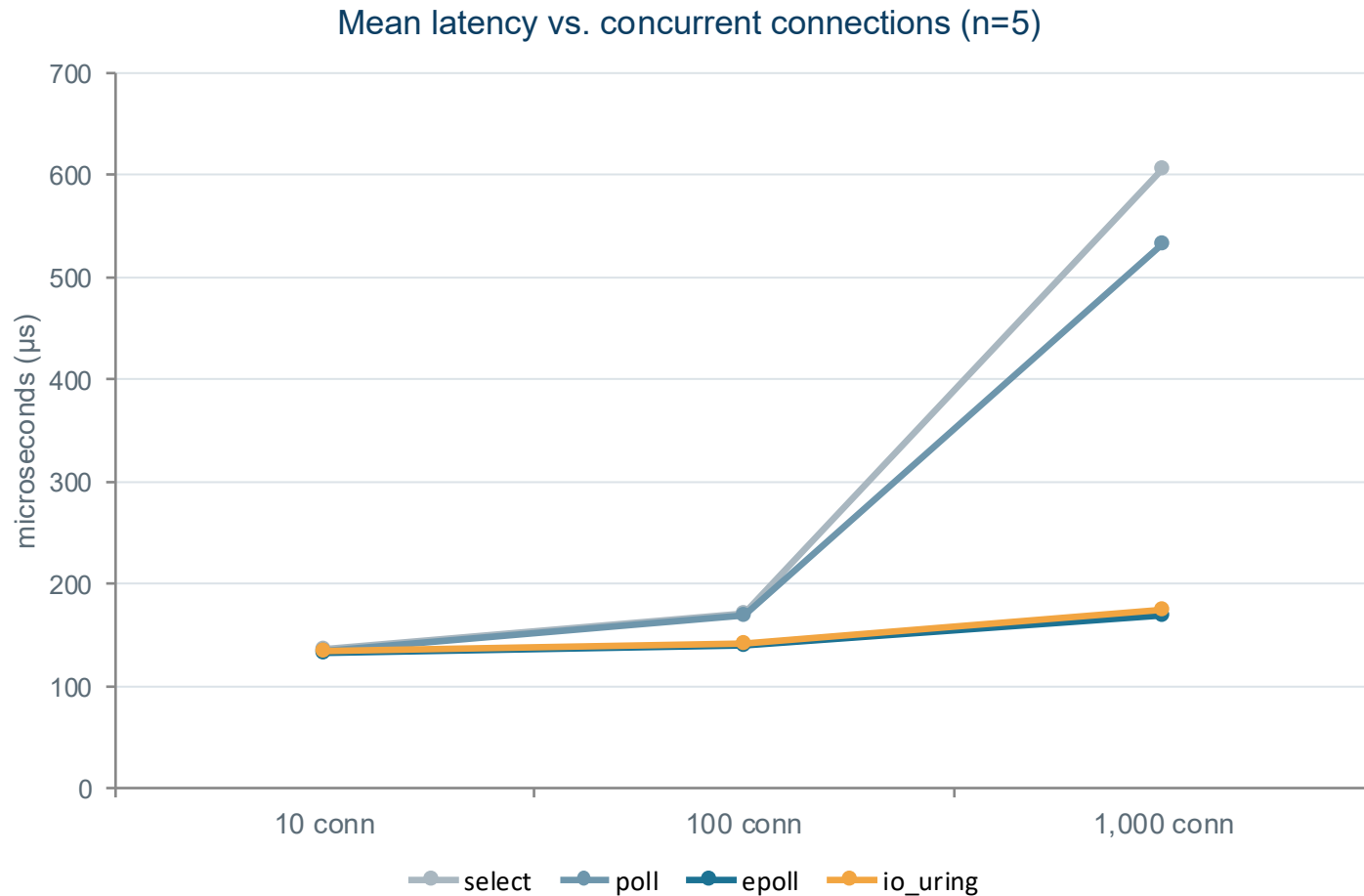
peak msg/s at 100 connections for epoll and io_uring — effectively tied.

~70% / 66%

throughput drop for select / poll between 100 and 1,000 connections.

RESULTS

Mean latency: flat for epoll/io_uring, steep for select/poll



605.8 μs

select mean latency at 1,000 connections — 4.5× its own 10-connection baseline.

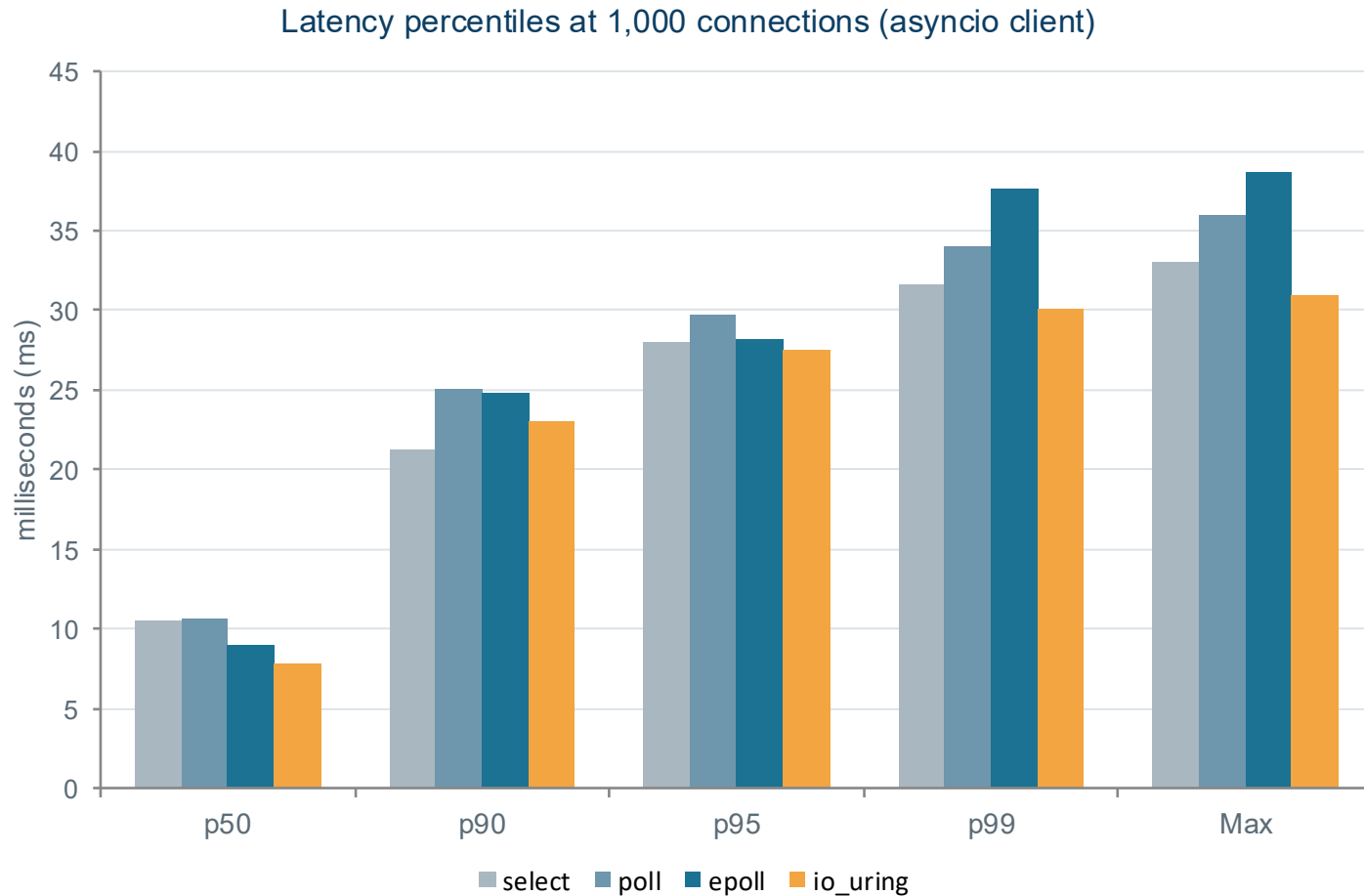
169.7 / 174.6 μs

epoll / io_uring mean latency at 1,000 connections — nearly flat across the whole load range.

Mean, here

This chart is the 5-repetition core benchmark (mean only) — true p50/p90/p95/p99 come from the supplementary pass, next.

True percentiles: what the mean was hiding



7.8 ms

io_uring's p50 — the lowest of the four, and it stays the most consistent all the way through to max.

37.6 / 38.8 ms

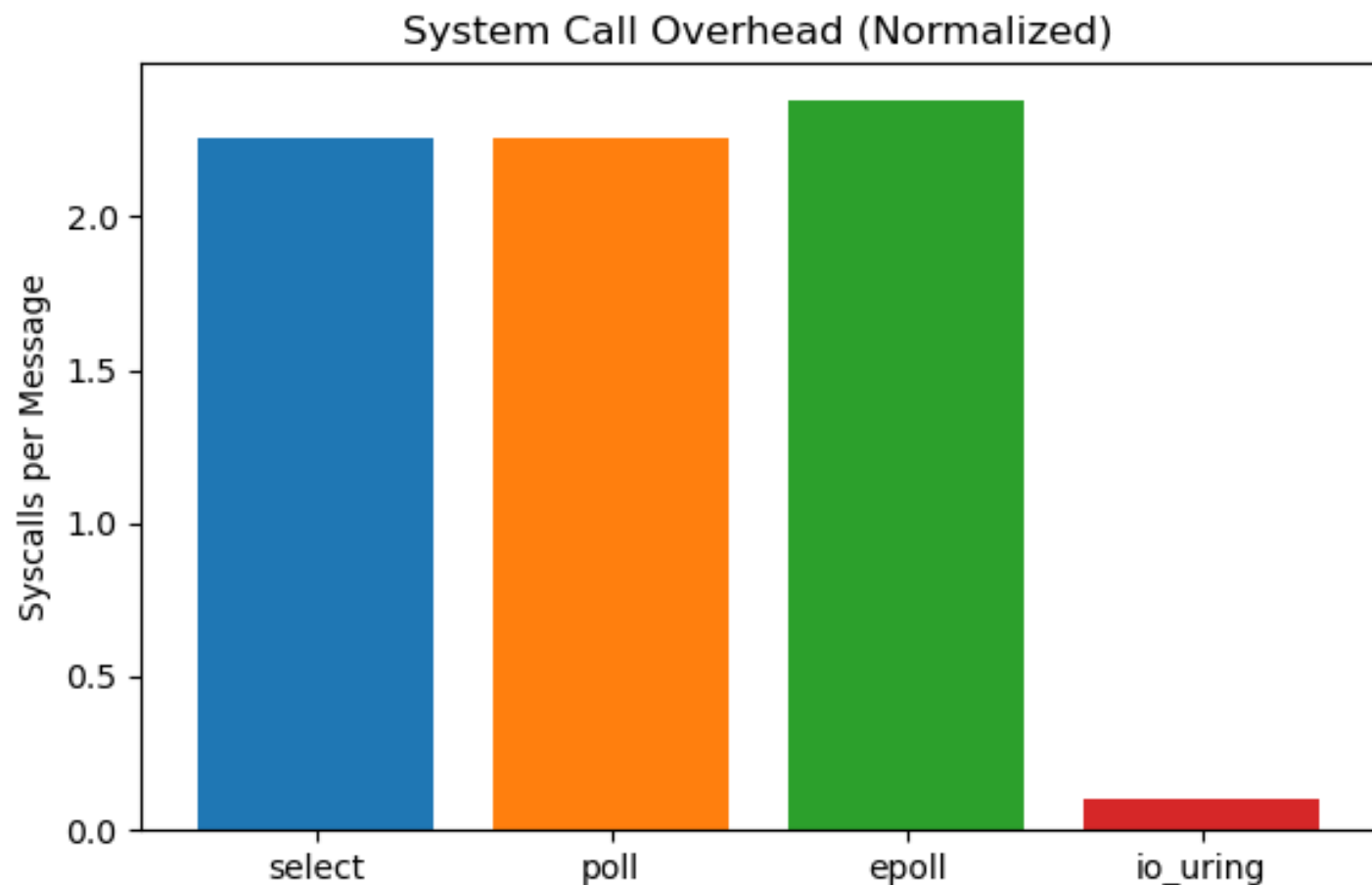
epoll's p99 / max — the highest tail of the four. io_uring has the lowest p95 and p99.

Ordering flips

The percentile ranking does not mirror mean latency: select has the lowest p90, while io_uring has the lowest p50, p95, and p99.

Measured, asyncio client, 1,000 connections, 10 msgs/conn, 1 rep — indicative of tail shape, not a precision match to the 5-rep core benchmark.

The syscall profile is where io_uring's story really is



0.101

io_uring syscalls per message — versus 2.256 for select, 2.254 for poll, and 2.382 for epoll.

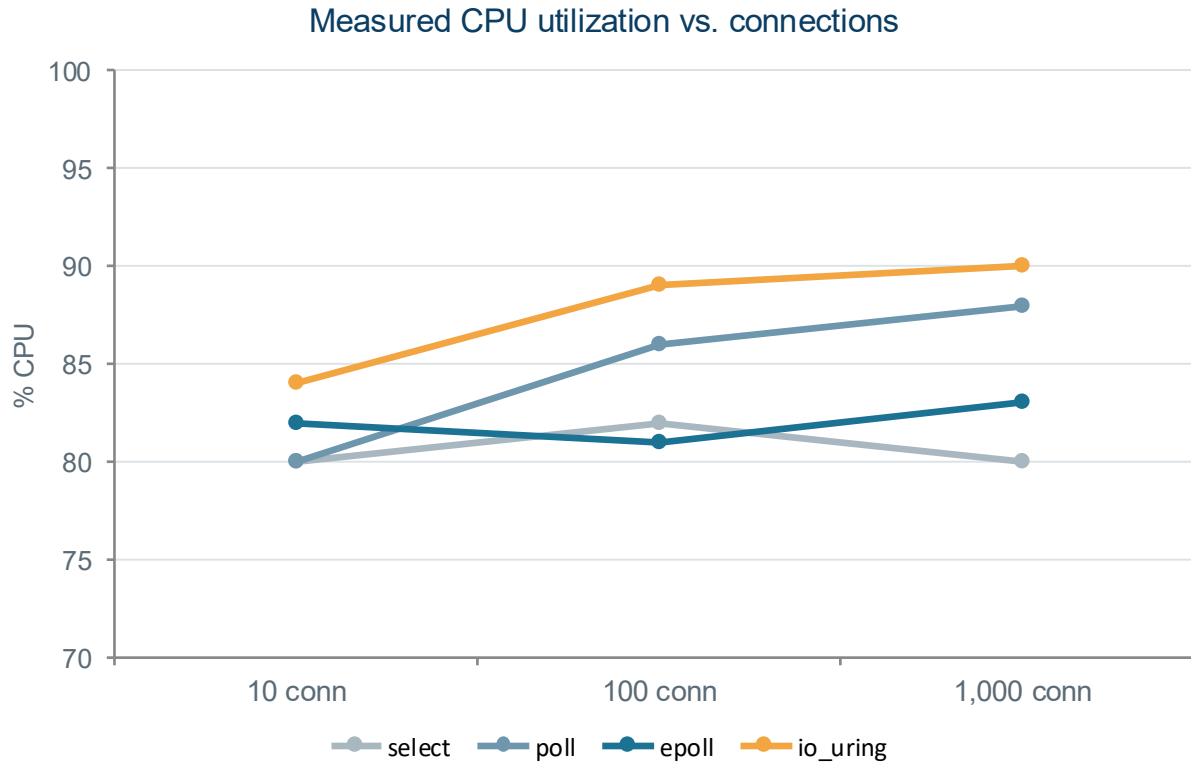
~23× lower

normalized syscalls/message for io_uring versus select/poll — the clearest architectural contrast in the report.

0.25 vs. 0.50

event-wait calls per completed message: epoll/io_uring vs. select/poll (Fig. 4 in the report).

What actually scales: CPU rises, memory doesn't



Measured via `/usr/bin/time -v`, single run per (engine, connection) pair.

Peak resident memory (RSS)

select	1,040 KB
poll	1,040 KB
epoll	1,040 KB
io_uring	1,072 KB

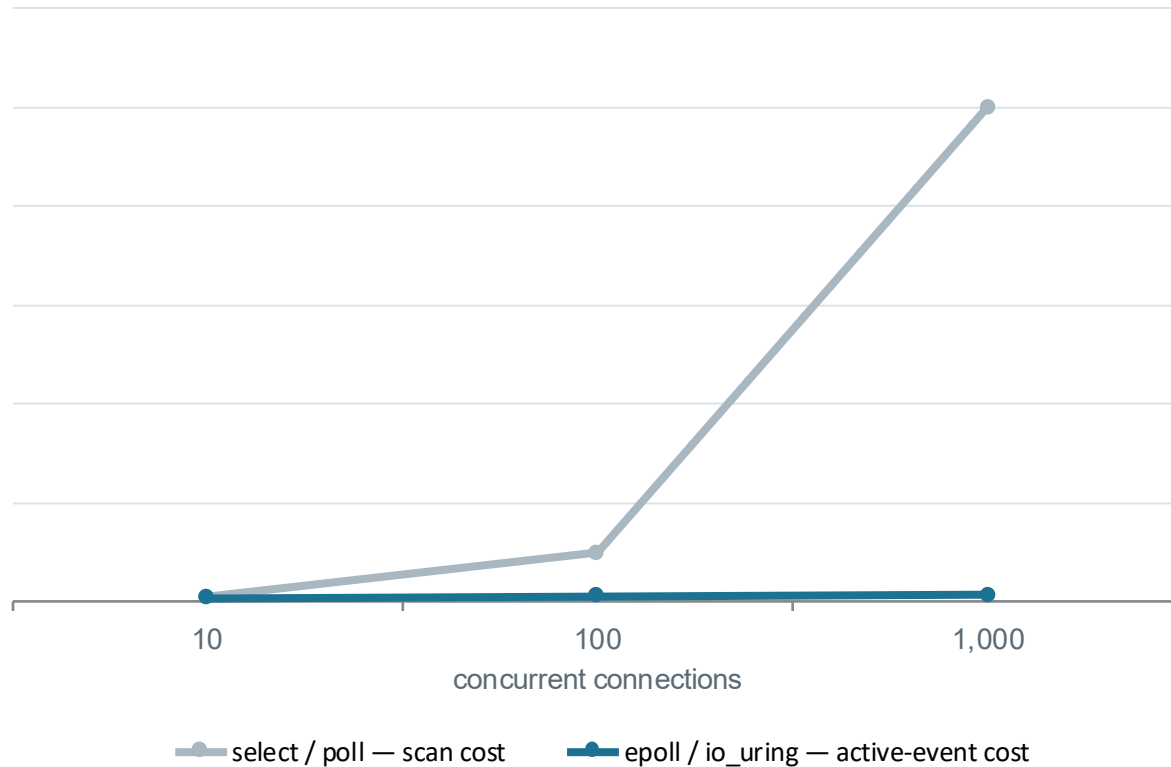
constant across 10 / 100 / 1,000 connections

CPU rises modestly with load for every engine, and `io_uring` is consistently the busiest (84%→90%) — it keeps draining completion batches rather than blocking in a wait call. `select` is the flattest (80%→80%, briefly 82% at 100), spending more of its time parked in one $O(N)$ syscall.

Memory tells a flatter story than expected: per-client state here is a small fixed-size struct indexed by `fd`, not a per-connection thread or buffer — so RSS does not scale with connection count for any engine, and the usual "epoll/io_uring use less memory" argument isn't visible as a measured effect in this implementation.

$O(N)$ scanning vs. $O(\text{ready})$ / $O(1)$: the real bottleneck

Relative per-call cost as connections scale (conceptual)



Illustrative — shape of the cost curve, not measured units.

select and poll both hand the kernel a full description of every watched descriptor on every call, and both the kernel and the application then scan that entire set — even when only a handful of descriptors are actually active. That per-call cost is $O(N)$ -class: it grows with how many connections are watched, not with how many are doing anything, which is exactly why throughput and syscall counts degrade sharply between 100 and 1,000 connections.

epoll's persistent, kernel-resident interest list breaks that link: `epoll_wait()` reports only descriptors that are ready, so wait cost tracks the active set rather than the watched set. `io_uring` goes further by batching submission and completion through shared rings, giving an effectively $O(1)$ amortized syscall cost per completed operation in this workload.

io_uring vs. epoll: fewer transitions, not a clean sweep

Why the ring buffer wins on user/kernel transitions

epoll still crosses the user/kernel boundary on each `epoll_wait()` call, plus separate `read()` and `write()` syscalls for socket I/O. `io_uring` instead uses shared submission and completion queues: the application can enqueue multiple operations and later drain multiple completions through `io_uring_enter()`. Batching is the mechanism — one crossing can carry many operations instead of one.

0.101

`io_uring` normalized syscalls per message — versus 2.382 for `epoll` on the reported profile.

But raw throughput doesn't tell a clean story

- ✦ At both 100 and 1,000 connections, `epoll`'s mean throughput is marginally higher than `io_uring`'s — well within their combined standard deviation.
- ✦ This implementation allows only one outstanding I/O operation per client, capping `io_uring`'s own pipelining headroom.
- ✦ The well-supported, defensible claim is architectural and syscall-based — `io_uring` drastically cuts user/kernel transitions — not a universal raw-throughput win over `epoll` on this small-payload, loopback workload.

Where this evaluation is still upfront about its edges

● Percentiles: smaller, separate run

True p50/p90/p95/p99/max (Fig. 5) come from a single-repetition, 10-msg/connection asyncio client at one concurrency level — tail-shape evidence, not a precision match to the 5-rep core benchmark.

● CPU/RSS: indicative, not averaged

`/usr/bin/time -v` wraps one short run per (engine, connection) pair, unlike throughput's 5-repetition average.

● Loopback only

Isolates event-loop behaviour from network variability, but says nothing about a real NIC, WAN link, or packet loss.

● select's descriptor ceiling

`FD_SETSIZE` is typically 1024, so the 1,000-connection point sits close to select's practical limit.

● Fairness normalization completed

A re-test standardized select and poll to infinite blocking, matching epoll/io_uring and removing the idle-timeout confound; it is therefore not listed as a remaining limitation.

● Profiling overhead

`strace -f -c` runs are 3–15× slower and used only for syscall counts, never mixed into throughput/latency figures.

CONCLUSION

A nuanced answer beats a clean-sounding one

- **select / poll** — Both pay repeated $O(N)$ descriptor-scanning cost on every wait call, regardless of how many watched connections are active — the direct, measured cause of their throughput collapse and latency growth beyond 100 connections.
- **epoll** — Avoids scanning inactive descriptors through a persistent, kernel-resident interest list ($O(\text{ready})$); select has the lowest p90, io_uring the lowest p50/p95/p99, and epoll the worst p99/max tail.
- **io_uring** — Removes the per-operation syscall boundary via shared submission/completion rings ($O(1)$ amortized), cutting traced syscalls by an order of magnitude — syscall efficiency, not an unconditional throughput win.
- **CPU & memory** — CPU rises modestly with load and asynchrony (io_uring highest, select flattest); RSS stays load-independent for all four, since per-client state is a small fixed struct, not a per-connection buffer or thread.

Four gaps flagged last time are now closed with measured data — true percentiles, CPU, memory, environment, and a fairness re-test. Remaining limitations are explicitly scoped: supplementary percentiles and CPU/RSS are single-run measurements, results are loopback-only, select is near its `FD_SETSIZE` ceiling, and profiling adds overhead.